Keystore research

I. Advanced Encryption Standard - AES

- Based on a design principle known as a substitution–permutation network, and is efficient in both software and hardware
- A variant of Rijndael.
- Fixed block size os 128 bits.
- A key size of 128, 192 or 256 bits (Rijndael *per se* is specified with block and key sizes that may be any multiple of 32 bits, with a minimum of 128 and a maximum of 256 bits).
- Number of transformation rounds that convert the input (plaintext) to the final output (ciphetext). Each round consists of several processing steps, including one that depends on the encryption key itself. A set of reverse rounds are applied to transform ciphertext back into the original plaintext using the same encryption key.
    - 10 rounds for 128-bit keys.
    - 12 rounds for 192-bit keys.
    - 14 rounds for 256-bit keys.
- AES can use one of several modes of operation (CBC, ECB, CTR, …) to allow encryption of data of arbitrary length. The recommended mode for the general case and for encrypting blockchain wallets is "CTR".
- The AES algorithm often is used along with a password-to-key derivation function, e.g. Scrypt(passwd) -> key or PBKDF2(passwd) -> key

II. Known attacks and bench

- By 2006, the best known attacks were on 7 rounds for 128-bit keys, 8 rounds for 192-bit keys, and 9 rounds for 256-bit keys.

=> There are known attacks against full AES. Using Biclique Cryptanalysis, full AES with a 128 bit key has been reduced to 126 bits of entropy, which seems to be the best result for key recovery attacks so far. The key space is, however, still impractical to search. At 1015 operations per second, which is the approximate computing power of a very large and expensive supercomputing cluster today, to find a key would still require an average time of 8.5 × 1022 seconds (2126 operations/1015 operations per second), or about 2.7 quadrillion years. The only known attacks against full AES also has the tiny problem of requiring petabytes of disk space. All in all, not very practical. The main reason it is stronger than it's predecessor, the DES

cipher, is due to it's larger key size. The fact that it is newer also reduce the window that cryptanalysts have had to come up with various attacks. Properly implemented AES has no known, practical attacks.

f	**Biclique**				
Rounds	Dimension	Δ^K bytes	∇^K bytes	Time	Memory
11-14	8	$\$12_2$	$\$13_5$	2^7	2^9

Matching					
g		Precomputation		Recomputation	
Rounds	v	Workload	Memory	SubBytes: forward	SubBytes: backward
1-10	$\#7_{12}$	2^7	2^9	0.625	4.8125

Total complexity					
Memory	$C_{biclique}$	$C_{precomp}$	C_{recomp}	$C_{falsepos}$	C_{full}
2^{13}	2^7	2^7	$2^{14.64}$	2^8	$2^{254.64}$

h1. Parameters of the key recovery in the full AES-256

Cipher	Attack/Result	Rounds	Data	Workload	Memory	Reference
AES-128	Multiset	6	2^{33}	2^{70}	2^{32}	[12]
	Collisions	7	2^{32}	2^{128}	2^{80}	[17]
	Partial sum	6	2^{35}	2^{44}	2^{32}	[14]
	Partial sum	7	$2^{128} - 2^{119}$	2^{120}	2^{32}	[14]
	Boomerang	6	2^{71}	2^{71}	2^{33}	[5]
	Impossible diff.	7	$2^{112.2}$	$2^{117.2}$	2^{109}	[27]
	Boomerang - RK	7	2^{97}	2^{97}	2^{34}	[9]
AES-192	Rectangle - RK	9	2^{64}	2^{143}	?	[18]
	Rectangle - RK	10	2^{125}	2^{182}	?	[24]
	Boomerang - RK	12	2^{116}	2^{169}	2^{145}	[9]
AES-256	Rectangle - RK	10	2^{114}	2^{173}	?	[4, 24]
	Subkey Diff.	10	2^{48}	2^{49}	2^{33}	
	Differential - RK[a]	14	2^{131}	2^{131}	2^{65}	[7]
	Boomerang - RK	14	$2^{99.5}$	$2^{99.5}$	2^{78}	[6]

h2. Summary of attacks on AES.

Cipher	Key size	Keys (Data)	Time	Memory	Preprocessing
DES	56	2^{14}	2^{28}	2^{28}	2^{42}
Triple-DES	168	2^{42}	2^{84}	2^{84}	2^{126}
Skipjack	80	2^{32}	2^{32}	2^{32}	2^{48}
AES	128	2^{32}	2^{80}	2^{56}	2^{96}
AES	192	2^{48}	2^{96}	2^{96}	2^{144}
AES	256	2^{85}	2^{170}	2^{85}	2^{170}
Any cipher	k	$2^{k/4}$	$2^{k/2}$	$2^{k/2}$	$2^{3k/4}$
Any cipher	k	$2^{k/3}$	$2^{2k/3}$	$2^{k/3}$	$2^{2k/3}$

h3. Comparison of TMD attacks on various ciphers.

One AES engine:	660K gates	2GHz clock speed	
One AES processor:	500 AES engines[a]	10^{12} AES ops/ s	30 US$[b]
CAESAR supercomputer:	$3 \cdot 10^{10}$ AES processors	$3 \cdot 10^{22}$ AES ops/s	1 trillion US$
AES chip production:	83 high capacity fabs	approx. 1 year	83 bln US$
Power consumption:	135 W per processor	$4\ \text{TW} = 4 \cdot 10^{12}$ W	
RKC Attack:	$9 \cdot 10^{29}$ AES ops	approx. 1 year	
Storage RKC:	2^{78} bytes		300 bln US$[c]
TMK Attack (2^{32} targets):	$0.8 \cdot 10^{29}$ pre-computation	30.6 days	
TMK Attack (2^{32} targets):	10^{24} ops per AES key	negligible	
Storage TMK:	2^{61} bytes		92 mln US$

[a] A 470 mm^2 die on 55nm TSMC CMOS process with 330M gates.
[b] This is a lower bound.
[c] Estimate for the next 5-10 years.

h4. Main characteristics of CAESAR and summary attack complexities.

- Other, Side-channel attacks do not attack the cipher as a black box, and thus are not related to cipher security as defined in the classical context, but are important in practice. They attack implementations of the cipher on hardware or software systems that inadvertently leak data. There are several such known attacks on various implementations of AES.

III. Algorithm evaluation

The worst purely cryptographic weakness is the password (1). The worst (and much worse) practical weaknesses are likely to be in the implementation (6) (7), and those (4) (5) that cryptographers dismiss as out of scope of encryption (the question's only stated objective). Candidates for weaknesses, from cryptographic (but still to consider) to implementation:

1. The password: it is hard to remember a password with over about 48 bits of entropy (that's about equivalent to a 16-digit number, like 3141592653589793 or 2718281828459045, and more than envisioned by the obligatory XKCD). Using N=262144=2^{18}, r=1, p=8=2^3, buys at the very least 18+3+1=22 bits (see this) compared to a straight SHA-256 hash followed by AES-128, I guess about 32 bits stronger, thus the key might be about 80-bit good, which is not entirely unbreakable by brute force but still quite secure.
2. AES-128: this has a 128-bit key, no known purely cryptanalytic attack better than brute force, and is thus much better than the password.
3. CTR mode: with AES's 128-bit block, the only way key-stream reuse could kick in would be a bad random number generator for the IV (see 6)
4. The size of the plaintext can be effortlessly deduced from the size of the ciphertext. This is explicitly *not* a theoretical weakness of encryption, since length is excluded from what a cipher is supposed to hide about plaintext. However, that can be a practical weakness. Say analysis of ciphertext concludes that it is 17267095423 octets; and there exists, somewhere in the prosecution's exhibits a file with exactly that size, demonstrably predating search and seizure of the ciphertext, and which mere detention is illegal.
5. Lack of detection of (intentional or accidental) ciphertext alterations, corresponding to predictable alteration in the plaintext: again this is explicitly *not* a theoretical weakness of encryption, but can allow some attacks if the adversary can alter the ciphertext (example: if the plaintext is an executable or PDF with a known fraction, that can allow changing what the executable does to about anything, or make the PDF display as about anything). Authenticated encryption (AES-GCM..) solves that.
6. Random number generator: if this is bad, or worse gets more or less stuck (obligatory XKCD and Dilbert which turn to reality all too often), that would enable recovering plaintext in many scenarios, including repeatedly enciphering different English text or email addresses with the same password.
7. A side channel, that is broadly speaking unforeseen or unmitigated leak of information in the implementation or usage context. There are so many:
    a. Password recovery from the password holder by exploiting a procedural goof (password on a postit note, master password list), rubber-hose cryptanalysis (obligatory XKCD), legal variants (involving use of expressions on the tune of *contempt of court*), bribery, chemical impregnation..
    b. Password recovery by shoulder surfing, key logger (software or hardware), camera, microphone (a good audio capture of the sound of keys when entering a password leaks a lot of info about the password, especially when correlated with known keyboard entry by same operator under same conditions)..

c. Straight compromise of plaintext as shown on screen (with TEMPEST variants of that), printed, while transmitted to a printer or remotely accessed by a network, or just "temporarily" stored..
d. Straight compromise of the OS of the machine doing encryption or decryption (getting temporary root access is enough, means in 7.7 are game to the attacker).
e. Cryptanalytic side channel at a distance, including DPA and electromagnetic variant DEMA, and hypothetically timing if the machine doing the encryption/decryption is accessed by a network and does not use AES instructions.
f. Cryptanalytic side channel from a process running on the same hardware (including another VM), including cache-based targeting for AES when not using AES instructions, or more general ones (Meltdown, Spectre) which are all the rage these days.
g. Exploitation of various software bugs; that's huge.

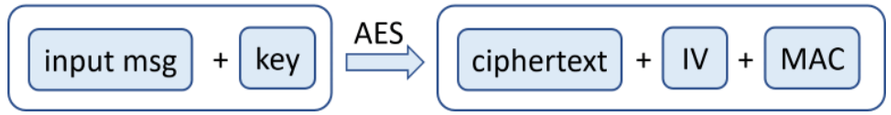
https://en.wikipedia.org/wiki/Advanced_Encryption_Standard

https://crypto.stackexchange.com/questions/55014/how-insecure-is-aes-128-ctr-to-encrypt-any-kind-of-data-using-the-ethereum-keyst
https://cryptobook.nakov.com/symmetric-key-ciphers/aes-cipher-concepts\

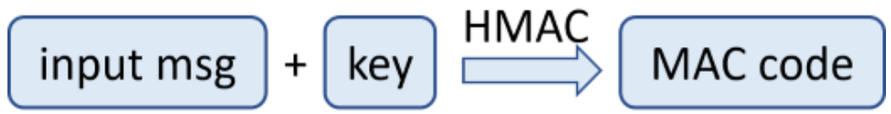
IV. AES Encryption Process

- Algorithm parameters are selected (e.g. AES, 128-bit, CTR mode + Scrypt + Scrypt parameters + MAC algorithm). These parameters can be hard-coded in the AES algorithm implementation source code or can be specified as input for the AES encrypt and decrypt. Always use the same parameters for encryption and decryption.
- The encryption key is derived from the encryption password using a key-derivation function (KDF), e.g. Scrypt (with certain parameters):

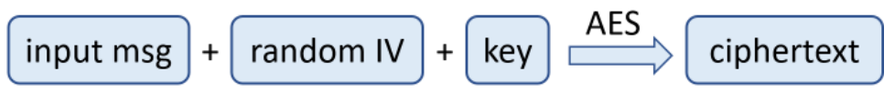
- The AES encryption scheme takes as input the input msg + the encryption key. It produces as output the ciphertext + the randomly generated IV (128-bit salt) + the MAC code:



- In case of authenticated encryption (e.g. AES-GCM), the MAC is already calculated automatically during the AES encryption process.
- If the encryption scheme is not authenticated encryption (e.g. AER-CTR), the MAC code is not calculated automatically by the AES encryption process and should be calculated additionally. The MAC code can be calculated from the input msg, using the encryption key (or some transformation of it) and some MAC function (like HMAC-SHA-256):



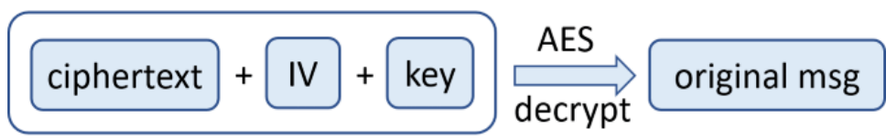
- The ciphertext is calculated through the AES encryption algorithm. It first generates a random salt (IV) and uses it to transform the input msg using the encryption key, through the AES cipher encryption logic:



– Finally, the encrypted output is generated. It holds the ciphertext + IV + MAC. Optionally, it holds also the algorithm settings.

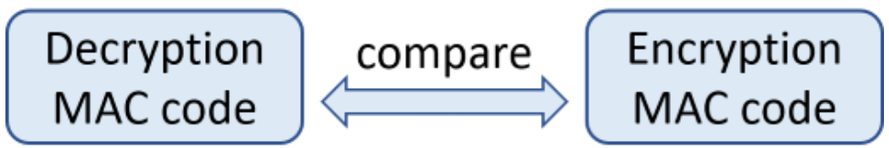
V. AES Decryption Process

– Initialize the same AES algorithm parameters for the decryption process, exactly like the ones used during the encryption.
– Use the decryption key and the IV from the encrypted message to decrypt the ciphertext using the AES algorithm decryption logic. The output is the original message (the input msg, which was previously passed through AES encryption):



    - In case of authenticated encryption (like AES-GCM), the integrated MAC code is verified during the decryption process.
    - In case of unauthenticated encryption (like AES-CTR), the MAC code should be calculated and verified additionally, as it is described in the next few step.
- Calculate HMAC of the original message (obtained during the decryption):

- Compare the encryption MAC (the MAC of the input message, before the encryption) with the decryption MAC (the MAC of the original message, recovered by the decryption):



    - If the MAC codes are the same, the decryption was correct and the original message is obtained.
    - If the MAC codes are different, the decryption was failed and the original message is not the obtained one. This may happen due to many reasons, most likely "wrong password". Other reasons: incorrect ciphertext, incorrect IV, incorrect algorithm settings, incorrect KDF function or KDF parameters, etc.

VI. UTC / JSON Keystore

Typically a UTC / JSON keystore holds the following data:

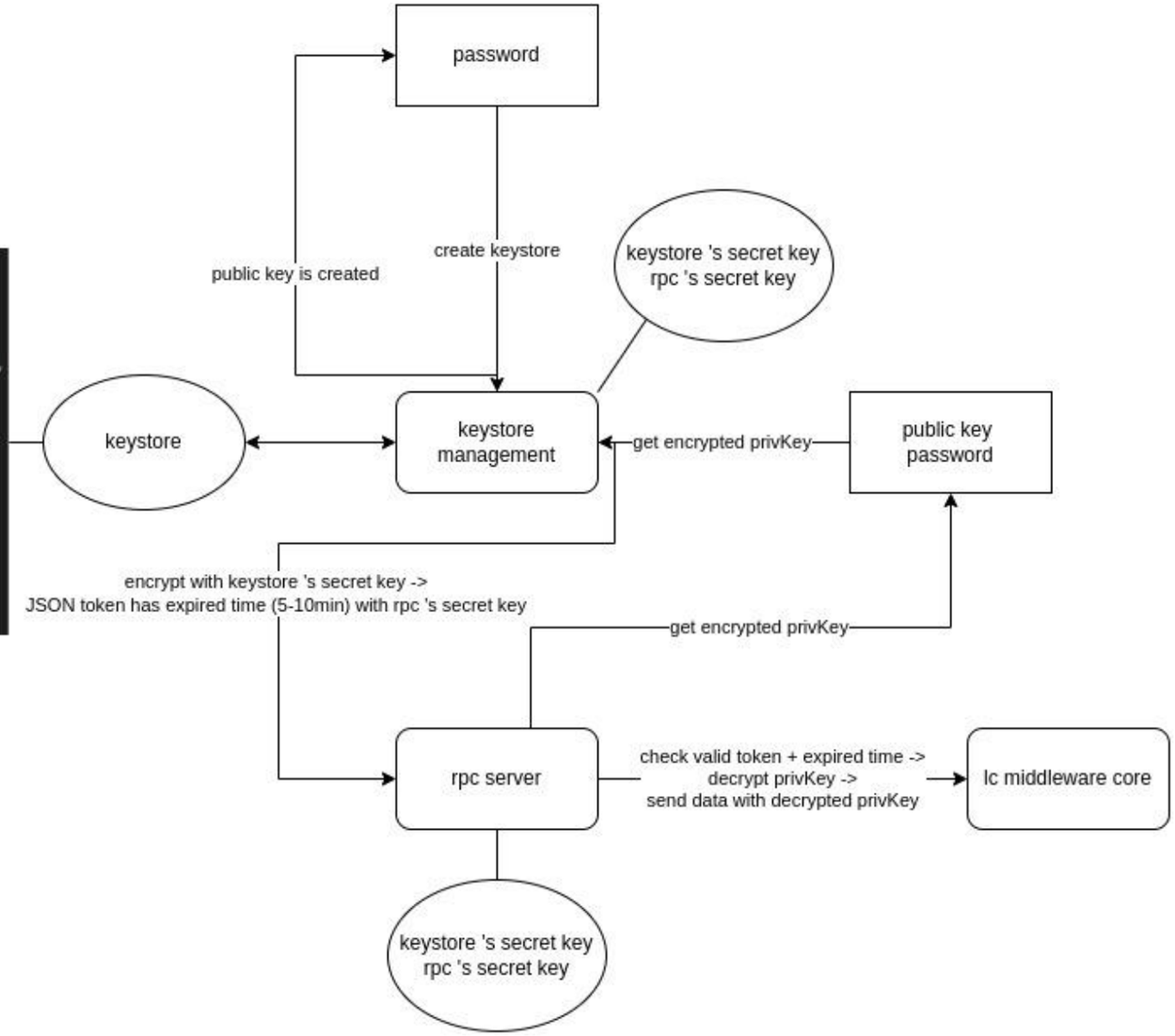
- Key-derivation function (KDF) used to transform the text-based wallet encryption password into an AES symmetric key, used to encrypt the wallet contents. Usually the KDF function is "scrypt".
    - The KDF parameters - the parameters used in the KDF function to derive the password (e.g. iterations count, salt, etc.)
- The ciphertext - the encrypted wallet content (typically holds an encrypted 256-bit private key).

- Symmetric cipher algorithm + its parameters, e.g. AES-128-CTR + initial vector (IV).
- MAC - message authentication code used (MAC) to check the message integrity after it is decrypted (to know whether the wallet decryption password was correct or not).
  - Ethereum calculates the MAC by calculating keccak-256 hash of the concatenations of the second-leftmost 16 bytes of the derived key together with the full ciphertext.
- Additional metadata: wallet format version, wallet unique id (uuid) and the blockchain address, controlled by this wallet.

By default the key-derivation function is scrypt and uses weak scrypt parameters (n=8192 cost factor, r=8 block size, p=1 parallelization), so it is recommended to use long and complex passwords to avoid brute-force wallet decryption attacks.

VII. Keystore management design

```
{
  version: 3,
  id: '0c8fc9d9-9b7d-4fa2-8742-d60d470eb436',
  address: '23b6972f7a888a7e61d612d141d515cd5bdde562',
  crypto: {
    ciphertext: '2c3f40d00bc05649c99a4678c3e0235a9b84d237836de97bbd0d95b23984ad8f',
    cipherparams: { iv: '1ab7d599b392930ead572b5c78c24594' },
    cipher: 'aes-128-ctr',
    kdf: 'scrypt',
    kdfparams: {
      dklen: 32,
      salt: 'e542c69e3cf3e970805a8b63869f8ad8ea3144b0fa7b7918ff7df678a5bfdc6c',
      n: 262144,
      r: 8,
      p: 1
    },
    mac: '798bbf2f2fa4ae7f601729aaa006b6dd0120b16b181bd319bbb83083e83894e4'
  }
}
```

Error! Hyperlink reference not valid.

IBM Keystore best practices

Last Updated: 2022-05-27

Employ security best practices to keep your keystores and master keys secure.

Procedure

- Use standard operating system access security protocols to keep your local keystore files and keystore configuration files safe.
- For local keystores and centralized keystores, use strong keystore passwords. The **gsk8capicmd_64** command has a parameter, -strong, that enforces stronger passwords. For more information, see: GSKCapiCmd Users Guide
- Rotate database master keys regularly using the **ADMIN_ROTATE_MASTER_KEY** procedure.

- For local keystores and centralized keystores, change keystore passwords regularly using the **gsk8capicmd_64** command with the -changepw parameter.
- Back up files at regular intervals, and every time the keystore changes (for example, when a key or certificate is added, a master key is rotated, or the password is changed.)
    - Back up local keystore files and keystore configuration files, and store the backed up files in a secure location.
    - If you are using a centralized keystore, use built-in features of the key manager to back up the keystore. See: Backing up IBM® Security Key Lifecycle Manager
- Do not delete keys from the keystore. Deleting keys from the keystore might result in the inability to access backups or transaction log files. Backups use the master key that is specified during the backup operation and are not affected by the ADMIN_ROTATE_MASTER_KEY procedure. Although the master key is rotated for primary transaction log files following a call to the ADMIN_ROTATE_MASTER_KEY routine, the change is not immediate, and access to the old master key is required for a period.